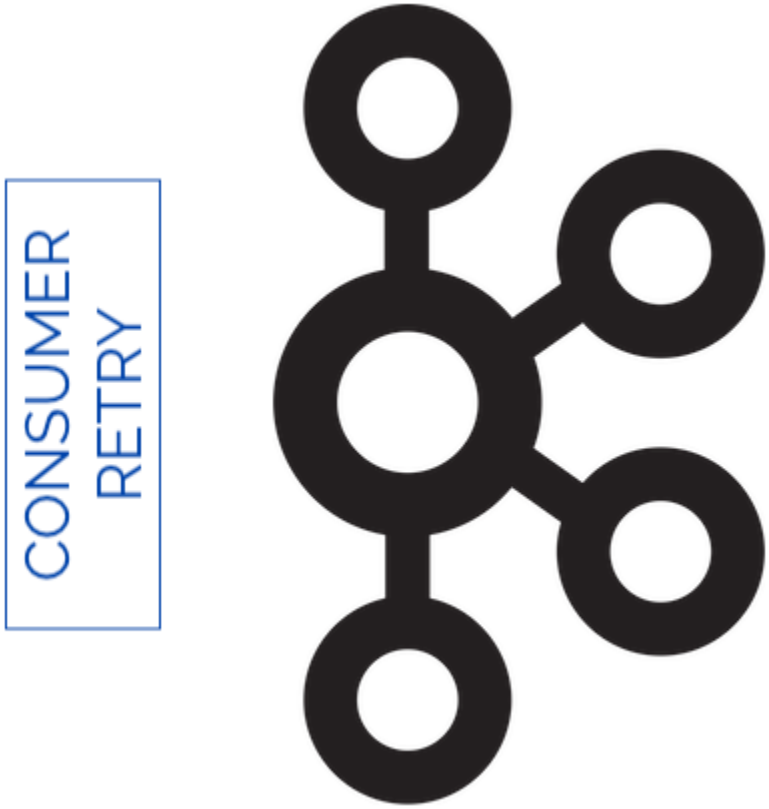
Kafka Consumer Retry

Rob Golder - October 2021



Introduction

The ability for an application to retry is essential in order to recover from transient errors such as network connection failures rather than simply failing the processing. When a flow is triggered by the consumption of an event, then the consumer should be configured to retry on such retryable exceptions. However there are a number of factors and pitfalls to consider with consumer retry, which this article explores.

Retryable Exceptions

There are many situations where retrying an action that threw an exception may result in a success. Examples include:

- A REST call to a third party service that returns a Bad Gateway (502) or a Service Unavailable (503) response, where the service may recover.
- An optimistic locking exception on a database write, where another process has updated the entity.
- A temporary database connection loss, or network connection error, both of which are considered transient errors.
- Internal Kafka exceptions like an offset not being available as it is lagging will also usually recover on retry.

If such exceptions are simply allowed to fail the process, perhaps writing an event to the dead-letter topic, then the system is brittle, and much time and effort will be spent subsequently trying to replay, or retrospectively fix, events that have failed.

Retry Considerations

Retryable exceptions should not simply be caught and retried indefinitely. Instead there are a number of issues to consider when configuring consumer retry, including:

- A topic partition is blocked while a message is being retried.
- Sufficient time should be allowed for transient errors to be retried.
- The risk of poison pill messages.
- If the Broker believes the consumer has died while it is retrying it will re-deliver the event, resulting in duplicate events.

Retry Period Factors

Analysis is required to determine how long retryable exceptions should be retried. If the retry period is too short then this may not allow sufficient time for the problem to be rectified. For example, a third party service might go offline for several hours.

This needs to be weighed up against the consideration that an event being retried will block the topic partition, as events behind it will not be processed in order to preserve Kafka's ordering guarantee. It is highly likely though that failing the message early in order to unblock the topic partition will not help as the next event will most likely fail for the same reason.

One option would be to retry retryable exceptions forever, until they succeed. However not having some kind of retry limit risks a poison pill message scenario. If the error scenario was never rectified, such as the third party system never coming back online, then the message would never be successfully processed, nor failed, as it continually retries. The application is not able to progress, the topic partition is blocked, and left like this the

message would likely be lost once it exceeded the Kafka topic retention period. A maximum retry period should almost always be in place.

A happy medium is required between too short a retry period and the retry forever risking a poison pill message, but here there is another risk to mitigate. The consumer polls the message from the broker, and if it has not completed processing it before the configured poll timeout, then the Kafka broker considers the consumer as failed and removes it from the consumer group. It then re-balances the consumer group, with a new consumer being assigned to the topic partition. The new consumer now also receives the same message as it polls the topic partition. If the original consumer successfully retries and processes the message, we now have a duplicate message in the system. Of course the second consumer could also suffer the same fate, and a third, or fourth, and so on, re-delivery could occur.

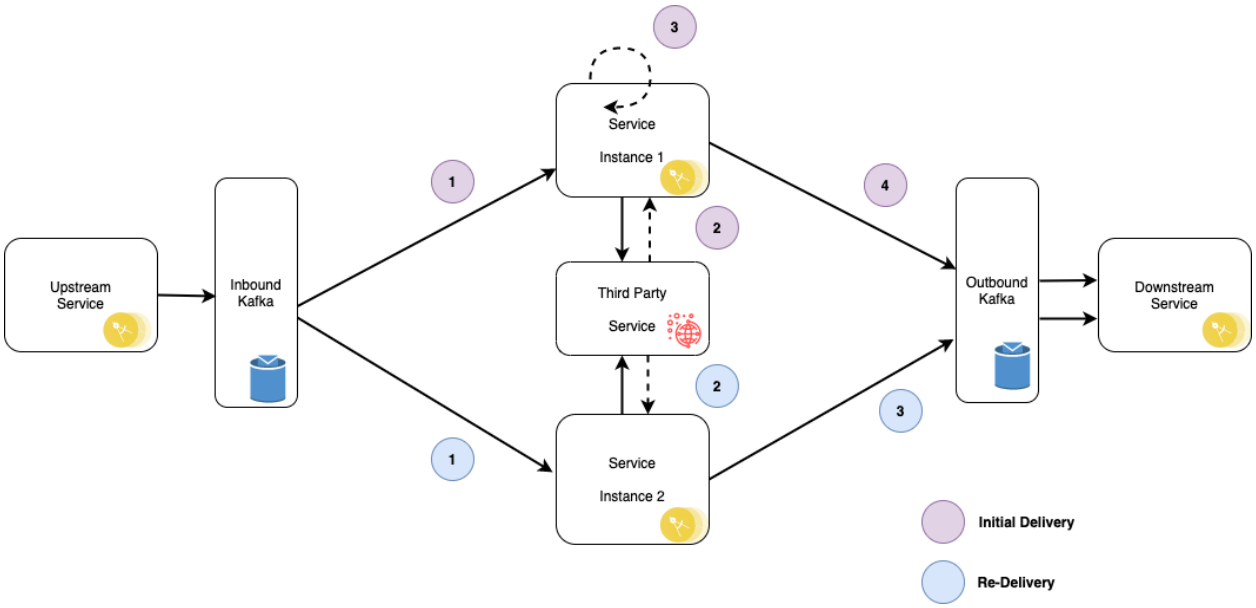
Stateless Retry

The Java Kafka client library offers stateless retry, with the Kafka consumer retrying a retryable exception as part of the consumer poll.

- Retries happen within the consumer poll for the batch.
- Consumer poll must complete before poll timeout, containing all retries, and total processing time (including REST calls & DB calls), retry delay and backoff, for all records in the batch.
- Default poll time is 5 minutes for 500 records in the batch. This only averages to 600ms per event.
- If poll time is exceeded this results in event duplication.
- Calculation of retries/time possible, but total retry duration will have to be short.

Stateless Retry Flow

The following diagram illustrates a service consuming an event, making a REST call to a third party service before publishing a resulting event to an outbound topic. There are two instances of the consuming service shown.



1 The first instance of the service consumes the event from the Kafka topic.

2 The service makes a REST call to a third party endpoint which fails with a 503.

3 The consumer retries the event multiple times within the poll, retrying the REST call each time.

The consumer max poll interval is exceeded while the service is retrying.

The broker re-balances the consumer group, assigning the consumer in the second service instance to the topic partition.

1 The second instance of the service consumes the event from the Kafka topic.

2 The service makes a REST call to a third party endpoint which succeeds.

4 The first service instance is still retrying and the REST call now succeeds, so a resulting event is published to the outbound topic.

3 The second service instance also publishes a resulting event to the outbound topic.

Any downstream services consume both resulting events. Although raised from a duplicate consumed event, these resulting events are distinct, so attempting to deal with the duplication issue downstream is not a recommended approach.

Calculating Stateless Retry Periods

There is little room available to safely retry for any period of time before duplicate messages are introduced into the system. Imagine a flow where a batch of events each result in a database update and an HTTP call to another service, each potentially taking up many seconds. To add in retry, and make use of exponential backoff, some very careful calculations must be made to ensure that the consume does not exceed the time out. If every message in a polled batch is being retried several times, there will be a very limited time for retry available. It will often simply not be possible to give sufficient time for transient exceptions to recover that otherwise could. Consider the third party service that is offline for several hours. These events must either be thrown back to the topic to potentially retry forever until success (with the risk of a poison pill scenario), or marked as failed and be sent to the dead letter topic.

Mitigation Strategies For Stateless Retry

There are options to mitigate this short retry period that stateless retry offers:

A) Polling timeout / batch size

- Polling timeouts can be increased and batch sizes reduced to give more time for retries within the consumer poll.
- If a long poll is put in place, and the consumer does die, the broker will not be aware of this until the poll eventually times out, leaving messages not being processed in the meantime.
- Reducing batch sizes comes at the cost of throughput.
- Practically the retry period will still be severely limited.

B) Retry topics

- Unblock topics by sending messages to retry to retry topic(s), with each subsequent retry topic having an increased back-off.
- Can add complexity, with more topics, more logic required, and more to test.
- Message ordering is lost, likely adding need for further logic and complexity.
- Spring retry topics are covered in detail in the article: [Kafka Consumer Non-Blocking Retry: Spring Retry Topics].
- A retry pattern using a retry topic when not developing using Spring-Kafka is covered in the article: [Kafka Consumer Non-Blocking Retry Pattern].

C) Idempotent Consumer pattern

- Accept that longer retries will result in duplicate messages and implement the Idempotent Consumer pattern to deduplicate messages.
- A mis-use of the pattern which should be used sparingly rather than as part of a retry strategy.
- All consumer instances will soon block while retrying, as the consumer group re-balances and the message is re-delivered to each consumer instance, using up database connections for deduplication (depending on deduplication approach).

D) Retry from the broker

- By throwing the Retryable exception rather than using the Kafka client library to retry, the message is not marked as consumed and is re-delivered in the next poll.
- The poll will not time out, so the message will not be duplicated.
- There is a huge risk of a poison pill message, as there is no means to limit the retry count.
- There is no ability to configure a retry delay or backoff.

Each of these mitigation strategies come at different costs which would need to be carefully weighed up if using stateless retry.

Stateful Retry

Solving Stateless Retry

The problems discussed here with stateless retry are solved by using stateful retry. When a message should be retried it is not marked as consumed so is picked up again on the next poll. As only one Consumer polls from any one topic partition, this same Consumer will re-receive the message to retry. The Spring Consumer tracks the retries for the message so knows once the retries have exhausted. This is the ‘state’ in stateful retry, but it is tracked in memory, there are no other components such as a database in play here. Retry can safely be configured for a long period of time, knowing that if still not successful at the end of that period then the message can be dead-lettered. There is nothing like the same level of consumer re-balancing / message duplication concerns that come with the stateless approach. The only caveat is that the longest delay between any retry should not exceed the poll timeout, as that would cause a re-balance and duplicate message delivery. However this is far more manageable than attempting to ensure that all retries for all events in a batch are completed within the poll timeout.

Spring Kafka & Stateful Retry

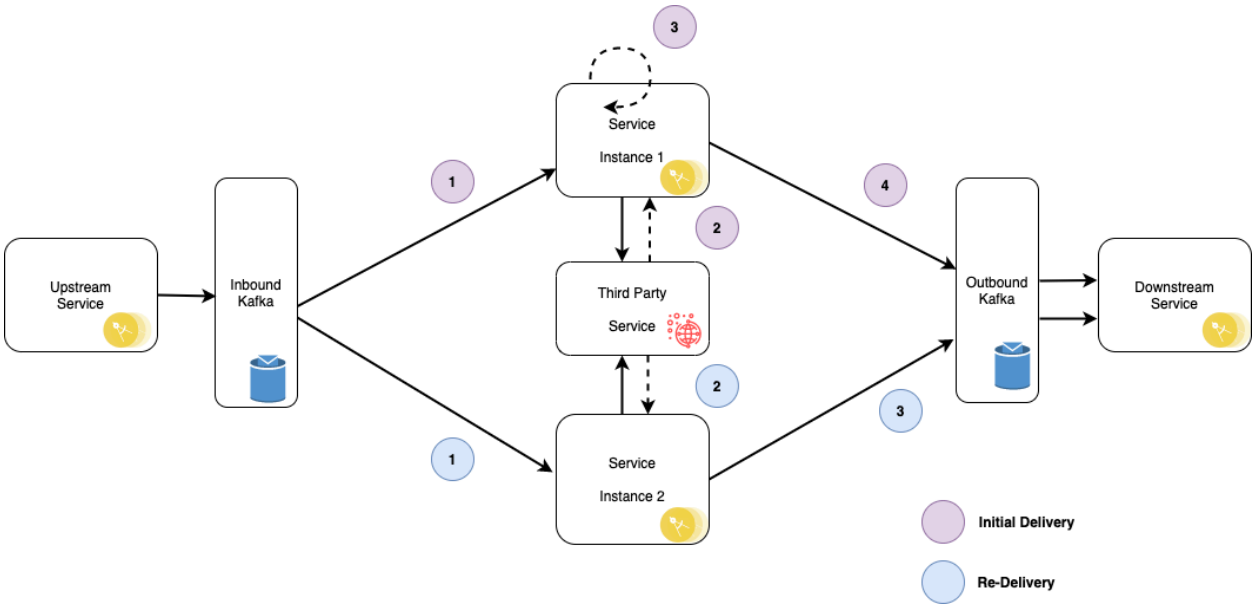
Stateful retry is not offered by the Java Apache client by itself, but is available as a configuration option out of the box with Spring Kafka, using the SeekToCurrentErrorHandler.

```java
@Bean
public ConcurrentKafkaListenerContainerFactory
kafkaStatefulRetryListenerContainerFactory(final ConsumerFactory
consumerFactory) {

   final SeekToCurrentErrorHandler errorHandler =
           new SeekToCurrentErrorHandler((record, exception) -> {
               // 10 seconds pause, 10 retries.
           }, new FixedBackOff(10000L, 10L));

   final ConcurrentKafkaListenerContainerFactory factory = new
ConcurrentKafkaListenerContainerFactory();
   factory.setConsumerFactory(consumerFactory);
   factory.setErrorHandler(errorHandler);
   return factory;
}
```

An additional feature of the SeekToCurrentErrorHandler is that those events within the batch that have successfully been processed prior to the event that results in a RetryableException being thrown are still able to be successfully marked as consumed, so are not themselves re-delivered too in the next poll.

Stateful Retry Flow

The same components from the stateless retry diagram are again drawn, but this time the consumer is configured to use stateful retry.



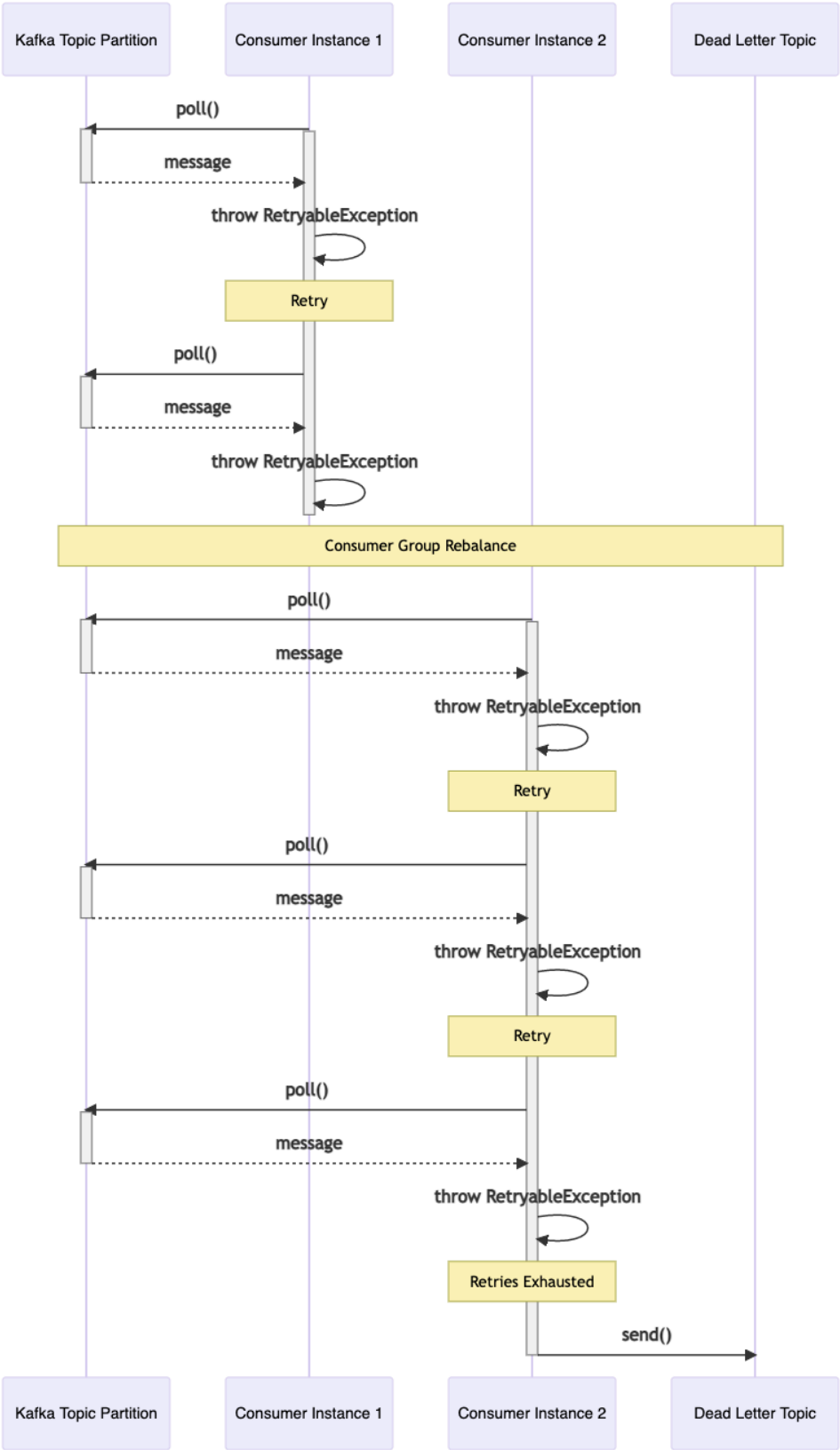
1. The first instance of the service consumes the event from the Kafka topic.
2. The service makes a REST call to a third party endpoint which fails with a 503.
3. The consumer re-polls the event from the broker for each retry.
4. The REST call eventually succeeds, so a resulting event is published to the outbound topic.

No poll timeout occurs, there is no consumer group re-balancing, and the message is not consumed by the second service instance.

Consumer Group Rebalance Impact

A consumer instance that is retrying a message using stateful retry could be unassigned from that topic partition during a consumer group rebalance. In this case the topic partition is assigned to another consumer instance which has no knowledge of the state of the retry. Therefore the retry attempts/period will be effectively reset.

In the following example 'max retry attempts' is configured to two. As a consumer group rebalance occurs after the first consumer instance has retried once and the partition is reassigned to the second consumer instance, three retries happen in total before the message is dead lettered.


Kafka Topic Partition
Consumer Instance 1
Consumer Instance 2
Dead Letter Topic
poll()
message
throw RetryableException
Retry
poll()
message
throw RetryableException
Consumer Group Rebalance
poll()
message
throw RetryableException
Retry
poll()
message
throw RetryableException
Retry
poll()
message
throw RetryableException
Retries Exhausted
send()
Kafka Topic Partition
Consumer Instance 1
Consumer Instance 2
Dead Letter Topic

For this to become an issue it would require both the unlikely event that frequent rebalances are occurring and the even more unlikely scenario that every consumer instance that is in turn retrying the message happens to be unassigned from that topic partition one after the other. The reassignment of the partition would then result in the retry attempts/period being reset each time, meaning that the total retry time could be theoretically unbounded. The message would eventually be expired from the topic once its age exceeded the retention period of the topic if the retries never successfully completed. So the message would never complete processing, rather than the expected retry attempts being exhausted and the message dead-lettered.

However, even if consumer group rebalances are occurring frequently, typically a healthy consumer instance that is retrying the message would not be unassigned from the topic partition, and so the state of the retry is retained. Unassignment of a partition is likely to only happen either if the consumer dies, or the instances are being scaled up so this partition is assigned to a new consumer.

Monitoring and Alerting

Retries should be monitored and alerts fired when retries are happening for long periods as this points to a possible system problem.

Consumer group rebalancing should also be monitored with alerting in place so that its frequency is understood. If it is occurring more frequently than expected (noting it does happen naturally when scaling consumers up and down) then this should be investigated. This would give confidence in the edge case that a message retrying via stateful retry is not at risk of expiring due to the retry attempts/period constantly being reset.

If the retention period of a topic is too short, and messages are being retried for long periods, be it with stateful or stateless retry, the messages blocked behind these could expire from the topic before they are ever consumed.

Conclusion

Stateless retry on a consumed event limits an application’s ability to retry for any useful length of time for most typical use cases, without the risk of message duplication.

Stateful retry solves this by not tying the total time for all retries to the consumer poll. Applications can be configured to retry transient errors for as long as required.

Source Code

The source code is available here:
https://github.com/lydtechconsulting/kafka-consumer-retry

The project demonstrates the difference in behaviour between consumers using stateless and stateful retry.

It contains SpringBoot integration tests that demonstrate the retry behaviour. They use an embedded Kafka broker and a wiremock to represent a third party service. This call to the third party service simulates transient errors that can be successful on retry.

It also contains component tests that demonstrate the retry behaviour. They use a dockerised Kafka broker and a dockerised wiremock to represent a third party service. This call to the third party service simulates transient errors that can be successful on retry. Two instances of the service are also running in docker containers.

More On Kafka Consumer Retry

- Kafka Consumer Non-Blocking Retry: Spring Retry Topics: provides an overview on blocking vs non-blocking retry, and how to apply Spring retry topics to achieve non-blocking retry.
- Kafka Consumer Non-Blocking Retry Pattern: the second article on non-blocking retry, detailing a pattern that can be applied when not using Spring Kafka.